

Optimizing the fleet's trajectory, not its next decision.

A supervisory layer that forecasts the fleet forward, simulates many coupled futures under constraints, and returns one policy for the existing stack to execute.

Aurelius sits above the scheduler. It does not replace Kubernetes, Slurm, or a serving router, and it never touches payloads. Each control period it forks the current fleet state into many forecasted futures, applies a coupled policy to each, advances them under SLA, capacity, power, and policy constraints, discards the ones that violate, and hands the single best safe policy back to the stack to run.

+724% average SLA-safe goodput per dollar

MEAN OF +698% / +718% / +755% (PJM / ERCOT / CAISO) VS A PRODUCTION-CLASS SCHEDULER BASELINE · UNCAPPED REPLAY OF PUBLIC PRODUCTION TRACES (~1.5M REQUESTS) · PARETO-SAFE AT ~84% FEWER GPU-HOURS

A number that size demands an explanation, and the report gives two. The **regime**: at production-scale load a fixed-policy scheduler degrades while a trajectory optimizer adapts. The **mechanism**: on the same benchmark, holding the model, objective, and gate fixed and changing only the search, the result climbs from about 3.6× the baseline on a fixed policy grid to the full headline once the search can represent coupled futures no grid contains. The number is not simulator optimism; it is where the search reaches.

00 Abstract

Conventional infrastructure stacks, including the unified control planes that consolidate a fleet into one view, optimize the next decision: the next placement, the next scale action, the next route, each chosen locally against the state visible now. Aurelius changes what gets optimized. It treats the fleet as a system with a trajectory, forecasts that trajectory forward, and optimizes the path rather than the step.

Concretely, it is not a forecaster and not a scheduler replacement. It is a supervisory layer that forks the current fleet state into many forecasted futures, applies coupled policies across the controls the stack already exposes, advances each future under hard constraints, kills the ones that violate, and returns one policy for the existing scheduler to execute.

Replaying public production traces uncapped, at full volume (about 1.5M requests across the PJM, ERCOT, and CAISO expensive-power windows), Aurelius averages **+724% higher SLA-safe goodput per dollar** than a production-class scheduler baseline (per market +698% / +718% / +755%), while holding SLA strictly better and using **~84% fewer GPU-hours**. That is a Pareto win, not goodput bought by spending more.

A result that large is only worth reading if it is explained and defended, so this report does both. §07 gives the regime that produces the magnitude, a fixed policy degrading under uncapped load while a trajectory optimizer adapts. §05 gives the mechanism, measured directly: on the exact uncapped benchmark, holding the model, the objective, and the safety gate identical and varying only the search the planner runs, the result climbs from a single-lever policy that cannot even complete the replay, to **3.6×** the baseline on a fixed joint-policy grid, to the full **8.3×** headline once the search can represent coupled futures no fixed grid contains. The value was not in the model or the objective. It was in the search over coupled futures.

Scope, stated once and meant throughout: these are deterministic simulated replays on public production traces. The headline ratio is load-dependent and is quoted only with its scope (§07). This is directional evidence that requires live-telemetry calibration before any production claim, not a deployment. Throughout, Aurelius runs read-only and needs only scheduler metadata, never payloads or model weights.

01 The shift is in what you optimize

Most schedulers, and the control planes above them, optimize a current-state or local objective: availability, fairness, utilization, latency, queue state, immediate capacity, immediate price. Each is reasonable in isolation. But the economic outcome of a decision is set by constraints that arrive **after** it is made: power and capacity headroom, congestion, memory and topology pressure, demand, and the price of power, which on wholesale markets can move several-fold within a day.

Consolidating telemetry into one control plane is necessary, and it is not the missing piece. A unified present answers the question what is the best next decision given everything I can see now. It does not answer what is the best sequence of decisions given what is about to happen. Those are different questions, and the second is where the compounding waste lives, because once a job lands its costs are largely locked in and an observe-then-decide loop cannot price a constraint it has not yet seen.

Aurelius closes that gap by moving from an **observe, decide** loop to an **observe, forecast, simulate, decide** loop. It is a different control architecture, not another scheduler, and it changes the question from the next decision to the trajectory.

03

The predictive world model

Aurelius maintains a persistent world model of the cluster: a forward model of the fleet, not a snapshot of free GPUs. It carries replica warm and cold state, server and rack capacity, placement and locality, queue dynamics, in-flight migrations, and the operator-cost basis, and it advances that state one control period at a time so a candidate decision is scored against the state it will actually meet.

Candidate policies are evaluated on read-only simulated future states, so scoring never mutates the live cluster. Every input is tagged by provenance, measured, trace-derived, benchmark-calibrated, or modeled, so no result silently treats a modeled value as a measured one.

04

Which futures get evaluated is a first-class step

Because the controls interact, Aurelius evaluates coupled bundles rather than tuning one knob, and which bundles it evaluates is itself a first-class step, not an afterthought. A forecast-guided generator proposes a focused set of high-value coupled bundles implied by the operating regime, always including the known-strong candidates so a high-value option is never silently dropped. The set is then searched in a way that captures cross-surface coupling, widening to more surfaces only when a decision is close and stopping early when the choice is clear.

Where exact enumeration is tractable, the bounded search is scored against it, so any regret is **measured** rather than assumed. That discipline is what surfaced the finding in §05, and it is what lets a several-fold headline be defended rather than asserted.

ACTIVE DECISION LEVERS SCORED AS COUPLED BUNDLES; EACH CHANGES THE PROJECTED ECONOMIC OUTCOME

Capacity policy · Capacity multiplier · Ordering · Admission · Batching · Routing · Prewarm · Placement · Migration · Precision · Clock / DVFS

The thesis of this report is a claim about where orchestration value actually sits, and it is not in a better next decision. Conventional schedulers, and the control planes above them, commit the next decision and re-plan when the following one arrives. Aurelius instead searches for the optimal **path** of coupled decisions, forecast many control periods ahead under constraints, and commits only the first step of the best path it finds. Almost nothing in production plans decisions this far down the road. That search, over forecasted trajectories rather than the state visible now, is where the value current orchestration leaves on the table is recovered.

The claim is testable, and the test is controlled. On the exact uncapped benchmark that produces the headline in §06, the forecasts, the simulator, the objective, the cost model, the constraint gates, and the baseline are held identical, and a single thing is varied: the search the planner is allowed to run. Every arm sees the same world and is judged the same way. Only how far and how well it searches the space of coupled futures changes, so the headline gap can be pinned to the search, not to a weaker baseline or a more generous model.

TABLE 1 · SAME UNCAPPED HARNESS, VARY ONLY THE SEARCH · SLA-SAFE GOODPUT/\$ RELATIVE TO THE REACTIVE BASELINE (MEAN ACROSS PJM, ERCOT, CAISO)

SEARCH STRATEGY	GP/\$ VS BASELINE	SLA VS BASELINE	NOTE
Reactive baseline (production scheduler)	1.00×	–	reference anchor
Single lever (clock only)	does not complete	–	replay cost blows up under load
Fixed joint-policy grid (24 policies)	3.62×	~4× lower	coupling, no search
Search the fixed grid (bounded beam)	4.85×	~4× lower	matches full grid enumeration
Coupled search beyond any grid	8.29×	lowest, ~18× lower	the +724% headline arm

Reproduces the published uncapped baseline byte-identically (130,538.91 / 130,178.26 / 130,000.36 gp/\$); the coupled-search arm reproduces the §06 headline within ~1%. Simulated; every completing arm also lowers the SLA-violation rate, the coupled arm the most (to ~0.002).

Read top to bottom, the ladder says three things:

- **Setting a coherent joint posture is worth about 3.6×**. Even a fixed grid of 24 policies that sets batching, capacity, precision, and clock together beats the reactive baseline several times over and cuts SLA violations roughly four-fold. Coupling beats reacting, before any search at all.
- **Searching that fixed grid harder saturates**. A bounded beam (4.85×) lands on the full exhaustive enumeration of the grid (4.84×). The bounded search already extracts everything the grid contains, at a fraction of the evaluations, so effort is not the bottleneck.
- **The near-doubling to 8.29× comes from representation, not effort**. The last and largest block of value, 4.85× to 8.29×, appears only when the search can represent coupled policies that live outside any fixed grid. That gap is the whole thesis, measured at headline load.

So the headline is not the baseline being weak (every joint arm beats it) and not the model being generous (the model is identical across arms). It is the search, and specifically the ability to represent and evaluate coupled futures no fixed grid contains. Where a window is small enough to enumerate the entire space, that same bounded search has matched the exhaustive optimum exactly, evidence that it captures the value available rather than stumbling onto a lucky point.

One exclusion, stated plainly. An aggressive low-precision (int4) mode scores higher still, but its quality cost is not yet modeled, so it is excluded from every headline and reported only as a labeled diagnostic ceiling. The headline-safe result uses the precision band whose quality can be defended.

06 The headline result: uncapped, production-scale replay

The headline replays the full selected windows with the per-period request cap removed, so each runs at its natural, production-scale volume: PJM ~577k, ERCOT ~443k, CAISO ~530k requests, about 1.5M in total. Both policies run the identical load. The bar is a **production-class scheduler baseline**, not a strawman. It is composed from the controls a modern GPU serving stack actually runs, each mapped to its primary source in the published literature: continuous (iteration-level) batching^{[1][2]}, SLA-aware ordering with deadline protection^[3], KV and prefix-cache-aware routing^[4], topology-aware placement^[5], backlog autoscaling with a warm-pool headroom^[6], and class-based admission^[3]. Because every lever is an established technique rather than an Aurelius invention, the composite is as close to a deployed scheduler as a public replay allows, and the only thing being measured is the coupled search Aurelius adds on top.

At that volume the naive SLA-aware policy does not complete: its no-batching, no-admission replay cost grows super-linearly and times out, so it is a reference only. The production-class scheduler and Aurelius both complete, which makes them the primary, production-comparable comparison: the policy built from real serving-stack components is the bar Aurelius has to beat.

TABLE 2 · UNCAPPED HIGH-LOAD REPLAY · AURELIUS VS PRODUCTION-CLASS SCHEDULER

MARKET	AURELIUS GP/\$	PRODUCTION GP/\$	Δ %	SLA (AUR.)	SLA (PROD.)	REQUESTS
PJM · expensive	1,041,842	130,539	+698.1%	0.0023	0.0382	576,912
ERCOT · expensive	1,064,602	130,178	+717.8%	0.0013	0.0447	442,716
CAISO · expensive	1,111,915	130,000	+755.3%	0.0018	0.0458	529,621
Average / total	–	–	+723.7%	–	–	1,549,249

gp/\$ and SLA-violation rates read directly from run output. Δ% is per market (Aurelius – production) ÷ production; the +724% headline is the equal-weighted average of the three (723.7%, shown unrounded). SLA violations fall from ~3.8–4.6% to ~0.1–0.2%.

TABLE 3 · GPU-HOURS AT THE SAME UNCAPPED LOAD

MARKET	AURELIUS GPU-H	PRODUCTION GPU-H	Δ GPU-HOURS
PJM · expensive	62.2	399.9	–84.4%
ERCOT · expensive	51.0	323.7	–84.3%
CAISO · expensive	60.0	391.5	–84.7%
Total	173.1	1,115.1	–84.5%

Same served requests both arms; the reduction is (production – Aurelius) ÷ production. Aurelius earns more SLA-safe goodput on far fewer GPU-hours because the coupled bundle packs more work per GPU-hour while holding SLA, which is why the win is Pareto and not paid for in spend.

07 How to read a several-fold number

A 7× gap over a competent scheduler is unusual, so the report is explicit about the two things that combine to produce it, and about where its edges are.

Two factors, multiplied. First, the regime: under uncapped, production-scale load a fixed-policy scheduler degrades super-linearly, serving heavy load worse until its goodput per dollar falls toward a floor. That floor is the ~130k baseline anchor, nearly identical across the three markets precisely because every fixed policy hits the same wall under the same load. It sets a low bar. Second, the search: above that bar, how far the optimizer reaches is set by the coupled search, which roughly doubles the result over a fixed joint policy (§05). The headline is the product of a degraded baseline and a search that separates most from it exactly when the fleet is stressed. Both are real, and neither alone is the whole number.

Where its edges are. The multiple is load-dependent. It moves with the per-period request volume and with how stressed the fleet is, so it is quoted only with its uncapped, full-window scope, never as a fleet constant. What does not depend on the load is the attribution in §05: the value comes from representing and searching coupled futures, and the direction holds across every load tested.

Read the several-fold number as two things at once: a stress signal, showing how far a fixed policy falls behind under load, and a mechanism, the coupled search that reaches futures no grid contains. It is not a fleet constant, and this report never quotes it as one.

08 What is actually new here

The individual pieces are not new, and this report does not claim they are. Forecasting, model-predictive control, digital twins of infrastructure, SLO-aware scheduling, and unified control planes all exist and are well understood. Aurelius's claim is narrower, and more durable for it.

What Aurelius optimizes is the fleet's **forecasted trajectory**, evaluated as coupled futures rather than independent next-decisions, and the binding constraint on the result is not the model or the objective but **which coupled futures the search can represent and evaluate**. The ablation in §05 is the evidence: on an identical stack, varying only the search moved the result from a policy that cannot complete the replay, to 3.6× the baseline on a fixed grid, to 8.3× once coupled futures outside any grid could be represented. That is a claim about where the value lives in fleet optimization, and it is testable on any fleet that keeps historical scheduler metadata.

09 Safety and the deployment path

Aurelius is **read-only and metadata-only**. It reads only the metadata a scheduler already exposes, job timing, resource requests, constraints, and never payloads, model weights, or model outputs. Candidate scoring runs on simulated futures, so it writes nothing to the live cluster. Unsafe candidates are rejected at the constraint gate before any recommendation exists, and when confidence is low the controller falls back deterministically to the stack's default.

The path from telemetry to any production change is staged and reversible: offline replay of historical decisions, then read-only shadow mode alongside the live scheduler, then a controlled rollout on workloads explicitly in scope. Aurelius owns the **decision**; the customer's platform owns the **execution**. The blast radius of any Aurelius fault is therefore a suboptimal-but-valid placement, never an unsafe action, because the existing scheduler is always the fallback.

10 Limitations

- Every number here is deterministic simulated replay of public traces: directional evidence of achievable savings, not a guarantee for any specific fleet, and not a production deployment.
 - The +724% headline is load-dependent. Part of the gap is a fixed-policy scheduler degrading under heavy uncapped load while Aurelius adapts, so it is reported only with its uncapped, full-window scope and never as a fleet constant.
 - Magnitudes are simulator-inferred and depend on the serving model's precision and batching bands. The most robust findings are the **direction** and the **attribution to search**, not any single percentage.
 - Aggressive low-precision is excluded from every headline until its quality cost is modeled.
 - Simulator fidelity must still be tested against real operator telemetry, which is the only way to isolate model error, and the largest gains appear during expensive-power windows.
 - Some controls are modeled but not yet active production levers, and several constraints are represented but not yet forecast inputs.
-

11 Validating the finding on a real fleet

The next step is not a bigger simulation, it is calibration against reality. Given read-only historical scheduler metadata from a production fleet, Aurelius would replay that fleet's own decisions, simulate the coupled counterfactuals, and produce an audited estimate of the SLA-safe goodput per dollar left on the table, with no production changes and no payload access.

That is the test of whether the trajectory-level result generalizes beyond public traces, and it is the fastest, lowest-risk way to find out. If it holds on a real fleet's own history, it is a new axis of optimization for that fleet. If it does not, the replay says so, cheaply, before anything touches production.

References

The production-class scheduler baseline (§06) is assembled from established production serving and cluster-scheduling techniques. These are cited for the realism of the baseline mechanisms, not for any Aurelius result.

- [1] Yu et al. "Orca: A Distributed Serving System for Transformer-Based Generative Models." OSDI 2022. Iteration-level (continuous) batching.
- [2] Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention" (vLLM). SOSP 2023. Continuous batching and KV-cache management.
- [3] Gujarati et al. "Serving DNNs like Clockwork: Performance Predictability from the Bottom Up." OSDI 2020. SLO-aware scheduling and admission.
- [4] Zheng et al. "SGLang: Efficient Execution of Structured Language Model Programs" (RadixAttention). 2024. Prefix / KV-cache-aware reuse.
- [5] SchedMD. "Slurm Workload Manager, Topology Guide." Topology-aware placement in production cluster schedulers.
- [6] Rządca et al. "Autopilot: Workload Autoscaling at Google." EuroSys 2020. Utilization-headroom autoscaling at production scale.